

REPORTE UNIFICADO DE CAMBIOS

Archivo añadido: src/main/java/com/mantenimiento/azul/report/UnifiedDiffReport.java

```
1 | package com.mantenimiento.azul.report;
2 |
3 | import com.itextpdf.kernel.colors.ColorConstants;
4 | import com.itextpdf.kernel.pdf.PdfDocument;
5 | import com.itextpdf.kernel.pdf.PdfWriter;
6 | import com.itextpdf.layout.Document;
7 | import com.itextpdf.layout.element.Cell;
8 | import com.itextpdf.layout.element.Paragraph;
9 | import com.itextpdf.layout.element.Text;
10 | import com.itextpdf.layout.properties.TextAlignment;
11 | import com.itextpdf.layout.element.Table;
12 | import com.mantenimiento.azul.model.ComparisonResult;
13 | import com.mantenimiento.azul.model.FileStats;
14 | import com.mantenimiento.azul.model.FormattedLine;
15 | import com.mantenimiento.azul.processor.DiffProcessor;
16 | import com.mantenimiento.azul.utils.LineFormatter;
17 |
18 | import java.io.IOException;
19 | import java.nio.file.Files;
20 | import java.nio.file.Path;
21 | import java.util.List;
22 | import java.util.Set;
23 |
24 | public class UnifiedDiffReport {
25 |
26 |     private final LineFormatter formatter = new LineFormatter();
27 |
```

```
28 | public void generate(ComparisonResult result, String oldPath, String newPath, String
outputPdf) throws IOException {
29 |
30 |     PdfWriter writer = new PdfWriter(outputPdf);
31 |     PdfDocument pdf = new PdfDocument(writer);
32 |     Document doc = new Document(pdf);
33 |
34 |     doc.add(new Paragraph("REPORTE UNIFICADO DE
CAMBIOS").setBold().setTextAlignment(TextAlignment.CENTER));
35 |
36 |     addRemovedFiles(result.getRemovedFiles(), doc, oldPath);
37 |     addAddedFiles(result.getAddedFiles(), doc, newPath);
38 |     addModifiedFiles(result.getModifiedFiles(), doc, oldPath, newPath);
39 |
40 |     doc.close();
41 | }
42 |
43 | private void addRemovedFiles(Set<String> removedFiles, Document doc, String oldPath)
throws IOException {
44 |     for (String file : removedFiles) {
45 |         Path path = Path.of(oldPath, file);
46 |
47 |         if (!Files.exists(path)) continue;
48 |
49 |         doc.add(new Paragraph("Archivo eliminado: " + file).setBold());
50 |         List<String> lines = Files.readAllLines(path);
51 |
52 |         for (int i = 0; i < lines.size(); i++) {
53 |             Text t = new Text(String.format("%4d | %s", i + 1,
lines.get(i))).setFontColor(ColorConstants.RED);
```

```
54 |         doc.add(new Paragraph().add(t));
55 |     }
56 |     doc.add(new Paragraph("\n"));
57 | }
58 | }
59 |
60 | private void addAddedFiles(Set<String> addedFiles, Document doc, String newPath)
throws IOException {
61 |     for (String file : addedFiles) {
62 |         Path path = Path.of(newPath, file);
63 |         if (!Files.exists(path)) continue;
64 |         doc.add(new Paragraph("Archivo añadido: " + file).setBold());
65 |         List<String> lines = Files.readAllLines(path);
66 |         for (int i = 0; i < lines.size(); i++) {
67 |             Text t = new Text(String.format("%4d | %s", i + 1,
lines.get(i))).setFontColor(ColorConstants.GREEN);
68 |             doc.add(new Paragraph().add(t));
69 |         }
70 |         doc.add(new Paragraph("\n"));
71 |     }
72 | }
73 |
74 | private void addModifiedFiles(Set<String> modifiedFiles, Document doc, String oldPath,
String newPath) throws IOException {
75 |     for (String file : modifiedFiles) {
76 |         Path oldF = Path.of(oldPath, file);
77 |         Path newF = Path.of(newPath, file);
78 |         if (!Files.exists(oldF) || !Files.exists(newF)) continue;
79 |
80 |         doc.add(new Paragraph("Archivo modificado: " + file).setBold());
```

```
81 |
82 |     List<String> oldLines = Files.readAllLines(oldF);
83 |     List<String> newLines = Files.readAllLines(newF);
84 |
85 |     DiffProcessor processor = new DiffProcessor(oldLines, newLines);
86 |     List<FormattedLine> diffs = processor.generateDiff();
87 |
88 |     for (FormattedLine line : diffs) {
89 |         for (Text t : formatter.formatLine(line)) {
90 |             doc.add(new Paragraph().add(t));
91 |         }
92 |     }
93 |
94 |     doc.add(new Paragraph("\n"));
95 | }
96 | }
97 |
98 | public void printStatsReport(List<FileStats> results, String projectPath) throws IOException
99 | {
100 |     int totalPhysicalLines = 0;
101 |     int totalLines = 0;
102 |
103 |     String projectName = Path.of(projectPath).getFileName().toString();
104 |
105 |     PdfWriter writer = new PdfWriter("reporte_estadisticas_" + projectName + ".pdf");
106 |     PdfDocument pdf = new PdfDocument(writer);
107 |     Document doc = new Document(pdf);
108 |
109 |     doc.add(new Paragraph("REPORTE DE ESTADÍSTICAS DE
110 | PROYECTO").setBold().setFontSize(16).setTextAlignment(TextAlignment.CENTER));
```

```
109 |     doc.add(new Paragraph("Programa: " + projectName).setBold().setFontSize(12));
110 |
111 |     // Definimos la tabla con 6 columnas
112 |     float[] columnWidths = {160f, 60f, 80f, 80f, 90f, 90f};
113 |     Table table = new Table(columnWidths);
114 |     table.setWidth(100);
115 |
116 |     // Encabezado
117 |     table.addCell(new Cell().add(new Paragraph("Clase").setBold()));
118 |     table.addCell(new Cell().add(new Paragraph("Métodos").setBold()));
119 |     table.addCell(new Cell().add(new Paragraph("LOC físicas Clase").setBold()));
120 |     table.addCell(new Cell().add(new Paragraph("Líneas Clase").setBold()));
121 |     table.addCell(new Cell().add(new Paragraph("LOC físicas
Programa").setBold()));
122 |     table.addCell(new Cell().add(new Paragraph("Líneas Programa").setBold()));
123 |
124 |     // Datos
125 |     for (FileStats stats : results) {
126 |         totalPhysicalLines += stats.physicalLines();
127 |         totalLines += stats.lines();
128 |
129 |         for (var cls : stats.classes()) {
130 |             table.addCell(cls.getName());
131 |             table.addCell(String.valueOf(cls.getMethodCount()));
132 |             table.addCell(String.valueOf(cls.getPhysicalLOC()));
133 |             table.addCell(String.valueOf(cls.getLines()));
134 |             table.addCell(""); // espacio en blanco
135 |             table.addCell("");
136 |         }
137 |     }
```

```
138 |  
139 |     // Fila de totales  
140 |     table.addCell(new Cell(1, 4).add(new Paragraph("Totales").setBold()));  
141 |     table.addCell(new Paragraph(String.valueOf(totalPhysicalLines)).setBold());  
142 |     table.addCell(new Paragraph(String.valueOf(totalLines)).setBold());  
143 |  
144 |     doc.add(table);  
145 |     doc.close();  
146 | }  
147 |  
148 | }
```

Archivo añadido: src/main/java/com/mantenimiento/azul/utils/FileUtils.java

```
1 | package com.mantenimiento.azul.utils;  
2 |  
3 | import java.io.IOException;  
4 | import java.nio.file.*;  
5 | import java.nio.file.attribute.BasicFileAttributes;  
6 | import java.util.HashSet;  
7 | import java.util.Set;  
8 |  
9 |  
10 | public class FileUtils {  
11 |     public static Set<String> collectJavaFiles(String projectPath) throws IOException {  
12 |         Set<String> javaFiles = new HashSet<>();  
13 |         Path root = Paths.get(projectPath).toAbsolutePath().normalize();  
14 |  
15 |         try {  
16 |             Files.walkFileTree(root, new SimpleFileVisitor<>() {
```

```
17 |         @Override
18 |         public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
19 |             if (file.toString().toLowerCase().endsWith(".java")) {
20 |                 javaFiles.add(root.relativeTo(file).toString());
21 |             }
22 |             return FileVisitResult.CONTINUE;
23 |         }
24 |     });
25 | } catch (IOException e) {
26 |     System.err.println("Error: " + e.getMessage());
27 | }
28 | return javaFiles;
29 | }
30 | }
31 |
```

Archivo añadido: src/main/java/com/mantenimiento/azul/utils/FileAnalyzer.java

```
1 | package com.mantenimiento.azul.utils;
2 |
3 | import java.io.IOException;
4 | import java.nio.file.FileVisitResult;
5 | import java.nio.file.Files;
6 | import java.nio.file.Paths;
7 | import java.nio.file.SimpleFileVisitor;
8 | import java.nio.file.attribute.BasicFileAttributes;
9 | import java.util.ArrayList;
10 | import java.util.List;
11 |
12 | import java.nio.file.Path;
```

```
13 |
14 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
15 | import com.mantenimiento.azul.model.FileStats;
16 | import com.mantenimiento.azul.processor.CodeProcessor;
17 |
18 | public class FileAnalyzer {
19 |     public static List<FileStats> analyze(String path, CodeProcessor processor) {
20 |         List<FileStats> results = new ArrayList<>();
21 |
22 |         try {
23 |             Files.walkFileTree(Paths.get(path), new SimpleFileVisitor<Path>() {
24 |                 @Override
25 |                 public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
26 |                     if (file.toString().endsWith(".java")) {
27 |                         try {
28 |                             results.add(processor.processFile(file));
29 |                         } catch (InvalidLineFormatException e) {
30 |                             System.err.println("Error en " + file + ": " + e.getMessage());
31 |                         } catch (IOException e) {
32 |                             System.err.println("No se pudo leer " + file);
33 |                         }
34 |                     }
35 |                     return FileVisitResult.CONTINUE;
36 |                 }
37 |             });
38 |         } catch (IOException e) {
39 |             System.err.println("Error al analizar el proyecto: " + e.getMessage());
40 |         }
41 |
```



```
42 |     return results;
43 | }
44 | }
```

Archivo añadido: src/main/java/com/mantenimiento/azul/utils/FileLoader.java

```
1 | package com.mantenimiento.azul.utils;
2 |
3 | import java.io.BufferedReader;
4 | import java.io.FileReader;
5 | import java.io.IOException;
6 | import java.util.ArrayList;
7 | import java.util.List;
8 |
9 | public class FileLoader {
10 |     public static List<String> readLines(String filename) throws IOException {
11 |         List<String> lines = new ArrayList<>();
12 |         BufferedReader reader = new BufferedReader(new FileReader(filename));
13 |         String line;
14 |         while ((line = reader.readLine()) != null) {
15 |             lines.add(line);
16 |         }
17 |         reader.close();
18 |         return lines;
19 |     }
20 | }
```

Archivo añadido: src/main/java/com/mantenimiento/azul/model/FormattedLine.java

```
1 | package com.mantenimiento.azul.model;
2 |
```

```
3 | public class FormattedLine {
4 |     public int lineNumber;
5 |     public String content;
6 |     public String changeType;
7 |     public boolean isContinuation;
8 |
9 |     public FormattedLine(int lineNumber, String content, String changeType, boolean
isContinuation) {
10 |         this.lineNumber = lineNumber;
11 |         this.content = content;
12 |         this.changeType = changeType;
13 |         this.isContinuation = isContinuation;
14 |     }
15 | }
```

Archivo añadido: src/main/java/com/mantenimiento/azul/utils/LineFormatter.java

```
1 | package com.mantenimiento.azul.utils;
2 |
3 | import com.itextpdf.kernel.colors.ColorConstants;
4 | import com.itextpdf.layout.element.Text;
5 | import com.mantenimiento.azul.model.FormattedLine;
6 |
7 | import java.util.ArrayList;
8 | import java.util.List;
9 |
10 | public class LineFormatter {
11 |
12 |     private final int MAX_CONTENT_LENGTH = 80;
13 |
14 |     public List<Text> formatLine(FormattedLine line) {
```

```
15 | List<Text> formattedTexts = new ArrayList<>();
16 |
17 | String content = line.content.stripLeading();
18 | String indentation = line.content.replaceAll("^(\\s*).*$", "$1");
19 | String comment = line.changeType != null ? " // línea " + line.changeType : "";
20 |
21 | int index = 0;
22 | boolean firstLine = true;
23 |
24 | while (index < content.length()) {
25 |     String prefix = firstLine ? String.format("%4d | ", line.lineNumber) : "    | ";
26 |     String basePrefix = prefix + indentation;
27 |     int availableLength = MAX_CONTENT_LENGTH - basePrefix.length();
28 |
29 |     int remainingLength = content.length() - index;
30 |     String part;
31 |
32 |     if (remainingLength > availableLength) {
33 |         int cutIndex = index + availableLength;
34 |         int lastSpace = content.lastIndexOf(" ", cutIndex);
35 |         if (lastSpace <= index) lastSpace = cutIndex;
36 |
37 |         part = content.substring(index, Math.min(lastSpace, content.length()));
38 |         index = lastSpace;
39 |     } else {
40 |         part = content.substring(index);
41 |         index = content.length();
42 |     }
43 |
```

```
44 |         String fullText = basePrefix + part + (firstLine && index == content.length() ? comment
    | : "");
45 |         Text lineText = new Text(fullText);
46 |         if (line.changeType != null) {
47 |             switch (line.changeType) {
48 |                 case "añadida":
49 |                     lineText.setFontColor(ColorConstants.GREEN);
50 |                     break;
51 |                 case "borrada":
52 |                     lineText.setFontColor(ColorConstants.RED);
53 |                     break;
54 |             }
55 |         }
56 |         formattedTexts.add(lineText);
57 |         firstLine = false;
58 |     }
59 |
60 |     return formattedTexts;
61 | }
62 | }
63 |
```

Archivo añadido: src/main/java/com/mantenimiento/azul/app/ProgramAnalyzer.java

```
1 | package com.mantenimiento.azul.app;
2 |
3 | import java.util.Scanner;
4 | import java.util.Set;
5 | import java.util.List;
6 |
7 | import com.itextpdf.io.exceptions.IOException;
```

```
8 | import com.mantenimiento.azul.checker.Checker;
9 | import com.mantenimiento.azul.checker.CheckerFactory;
10 | import com.mantenimiento.azul.comparator.VersionComparator;
11 | import com.mantenimiento.azul.model.ComparisonResult;
12 | import com.mantenimiento.azul.model.FileStats;
13 | import com.mantenimiento.azul.processor.CodeProcessor;
14 | import com.mantenimiento.azul.report.UnifiedDiffReport;
15 | import com.mantenimiento.azul.utils.FileAnalyzer;
16 | import com.mantenimiento.azul.utils.FileUtils;
17 |
18 | public class ProgramAnalyzer {
19 |     private final Scanner scanner = new Scanner(System.in);
20 |
21 |     public ProgramAnalyzer() {
22 |     }
23 |
24 |     public void run(){
25 |         try {
26 |             while (true) {
27 |                 String oldVersionPath = prompt("Introduzca la ruta de la versión anterior: ");
28 |                 String newVersionPath = prompt("Introduzca la ruta de la nueva versión: ");
29 |
30 |                 Set<String> oldFiles = FileUtils.collectJavaFiles(oldVersionPath);
31 |                 Set<String> newFiles = FileUtils.collectJavaFiles(newVersionPath);
32 |
33 |                 ComparisonResult result = VersionComparator.compare(oldFiles, newFiles,
oldVersionPath, newVersionPath);
34 |                 generateReport(result, oldVersionPath, newVersionPath);
35 |                 printComparisonResults(result);
36 |
```

```
37 |         runCodeChecks(oldVersionPath);
38 |         runCodeChecks(newVersionPath);
39 |
40 |         if (!prompt("¿Desea analizar otra ruta? (y/n): ").matches("(?i)y|yes")) break;
41 |     }
42 | } catch (Exception e) {
43 |     System.err.println("Error: " + e.getMessage());
44 | } finally {
45 |     this.scanner.close();
46 | }
47 |
48 | }
49 |
50 | private String prompt(String message) {
51 |     System.out.print(message);
52 |     return this.scanner.nextLine().trim();
53 | }
54 |
55 | private void generateReport(ComparisonResult result, String oldPath, String newPath)
throws java.io.IOException {
56 |     try {
57 |         new UnifiedDiffReport().generate(result, oldPath, newPath, "cambios_unificados.pdf");
58 |         System.out.println("PDF unificado generado: cambios_unificados.pdf");
59 |     } catch (IOException e) {
60 |         System.err.println("Error generando el PDF unificado: " + e.getMessage());
61 |     }
62 | }
63 |
64 | private void printComparisonResults(ComparisonResult result) {
65 |     System.out.println("\n=== Comparación de Versiones ===");
```

```

66 |     printSection("Archivos nuevos", result.getAddedFiles(), "A");
67 |     printSection("Archivos eliminados", result.getRemovedFiles(), "D");
68 |     printSection("Archivos modificados", result.getModifiedFiles(), "M");
69 |     printSection("Archivos sin cambios", result.getUnchangedFiles(), "=");
70 | }
71 |
72 | private void printSection(String title, Set<String> files, String marker) {
73 |     System.out.println("\n" + title + " (" + files.size() + "):");
74 |     files.forEach(p -> System.out.println(" [" + marker + "] " + p));
75 | }
76 |
77 | private void runCodeChecks(String path) throws java.io.IOException {
78 |     Checker checkerChain = CheckerFactory.createCheckerChain();
79 |     CodeProcessor processor = new CodeProcessor(checkerChain);
80 |     List<FileStats> results = FileAnalyzer.analyze(path, processor);
81 |     if (!results.isEmpty()) {
82 |         UnifiedDiffReport report = new UnifiedDiffReport();
83 |         report.printStatsReport(results, path);
84 |     }
85 | }
86 | }

```

Archivo añadido: src/main/java/com/mantenimiento/azul/checker/CheckerFactory.java

```

1 | package com.mantenimiento.azul.checker;
2 |
3 | public class CheckerFactory {
4 |     public static Checker createCheckerChain() {
5 |         Checker parenthesesChecker = new ParenthesesChecker();
6 |         Checker leftCurlyBraceChecker = new LeftCurlyBraceChecker();

```

```
7 |     Checker multiInstanceChecker = new MultiInstanceChecker();
8 |     Checker endBreakChecker = new EndBreakChecker();
9 |
10 |     parenthesesChecker.setNext(leftCurlyBraceChecker);
11 |     leftCurlyBraceChecker.setNext(multiInstanceChecker);
12 |     multiInstanceChecker.setNext(endBreakChecker);
13 |
14 |     return parenthesesChecker;
15 | }
16 | }
```

Archivo añadido: src/main/java/com/mantenimiento/azul/processor/DiffProcessor.java

```
1 | package com.mantenimiento.azul.processor;
2 |
3 | import java.util.*;
4 |
5 | import com.mantenimiento.azul.model.FormattedLine;
6 |
7 | public class DiffProcessor {
8 |     private final List<String> oldLines;
9 |     private final List<String> newLines;
10 |
11 |     public DiffProcessor(List<String> oldLines, List<String> newLines) {
12 |         this.oldLines = oldLines;
13 |         this.newLines = newLines;
14 |     }
15 |
16 |     public List<FormattedLine> generateDiff() {
17 |         List<FormattedLine> result = new ArrayList<>();
```



```
18 |     int lineNumber = 1;
19 |     int oldIndex = 0, newIndex = 0;
20 |
21 |     while (oldIndex < oldLines.size() || newIndex < newLines.size()) {
22 |         String oldLine = oldIndex < oldLines.size() ? oldLines.get(oldIndex) : null;
23 |         String newLine = newIndex < newLines.size() ? newLines.get(newIndex) : null;
24 |
25 |         if (Objects.equals(oldLine, newLine)) {
26 |             result.add(new FormattedLine(lineNumber++, oldLine, null, false));
27 |             oldIndex++;
28 |             newIndex++;
29 |         } else {
30 |             boolean handled = false;
31 |
32 |
33 |             if (oldLine != null && (newLine == null || !newLines.contains(oldLine))) {
34 |                 result.add(new FormattedLine(lineNumber, oldLine, "borrada", false));
35 |                 oldIndex++;
36 |                 handled = true;
37 |             }
38 |
39 |             if (newLine != null && (oldLine == null || !oldLines.contains(newLine))) {
40 |                 result.add(new FormattedLine(lineNumber, newLine, "añadida", false));
41 |                 newIndex++;
42 |                 handled = true;
43 |             }
44 |
45 |             if (handled) {
46 |                 lineNumber++;
```

```
47 |         } else {
48 |             oldIndex++;
49 |             newIndex++;
50 |             lineNumber++;
51 |         }
52 |     }
53 | }
54 |
55 |     return result;
56 | }
57 | }
58 |
```

Archivo añadido: src/main/java/com/mantenimiento/azul/model/ComparisonResult.java

```
1 | package com.mantenimiento.azul.model;
2 |
3 | import java.util.HashSet;
4 | import java.util.Set;
5 |
6 | public class ComparisonResult {
7 |     private Set<String> addedFiles = new HashSet<>();
8 |     private Set<String> removedFiles = new HashSet<>();
9 |     private Set<String> unchangedFiles = new HashSet<>();
10 |     private Set<String> modifiedFiles = new HashSet<>();
11 |
12 |     public Set<String> getAddedFiles() {
13 |         return addedFiles;
14 |     }
15 |
```

```
16 | public void setAddedFiles(Set<String> addedFiles) {
17 |     this.addedFiles = addedFiles;
18 | }
19 |
20 | public Set<String> getRemovedFiles() {
21 |     return removedFiles;
22 | }
23 |
24 | public void setRemovedFiles(Set<String> removedFiles) {
25 |     this.removedFiles = removedFiles;
26 | }
27 |
28 | public Set<String> getUnchangedFiles() {
29 |     return unchangedFiles;
30 | }
31 |
32 | public void setUnchangedFiles(Set<String> unchangedFiles) {
33 |     this.unchangedFiles = unchangedFiles;
34 | }
35 |
36 | public Set<String> getModifiedFiles() {
37 |     return modifiedFiles;
38 | }
39 |
40 | public void setModifiedFiles(Set<String> modifiedFiles) {
41 |     this.modifiedFiles = modifiedFiles;
42 | }
43 | }
44 |
```

Archivo modificado: src/main/java/com/mantenimiento/azul/Main.java

```
1 | package com.mantenimiento.azul;

3 | import com.mantenimiento.azul.checker.MultiInstanceChecker; // línea borrada
3 | import com.mantenimiento.azul.app.ProgramAnalyzer; // línea añadida
4 | import com.mantenimiento.azul.checker.ParenthesesChecker; // línea borrada
5 | import com.mantenimiento.azul.comparator.VersionComparator; // línea borrada
6 | import com.mantenimiento.azul.checker.Checker; // línea borrada
7 | import com.mantenimiento.azul.checker.EndBreakChecker; // línea borrada
8 | import com.mantenimiento.azul.checker.LeftCurlyBraceChecker; // línea borrada
9 | import com.mantenimiento.azul.processor.CodeProcessor; // línea borrada
10 | import com.mantenimiento.azul.exception.InvalidLineFormatException; // línea borrada
11 | import com.mantenimiento.azul.model.FileStats; // línea borrada
13 | import java.io.IOException; // línea borrada
14 | import java.nio.file.*; // línea borrada
15 | import java.nio.file.attribute.BasicFileAttributes; // línea borrada
16 | import java.util.ArrayList; // línea borrada
17 | import java.util.HashSet; // línea borrada
18 | import java.util.List; // línea borrada
19 | import java.util.Scanner; // línea borrada
20 | import java.util.Set; // línea borrada

23 |     ProgramAnalyzer analyzer = new ProgramAnalyzer(); // línea añadida
24 |     analyzer.run(); // línea añadida

27 |     Scanner scanner = new Scanner(System.in); // línea borrada
28 |     String projectPath = args.length > 0 ? args[0] : "."; // línea borrada
30 |     while (true) { // línea borrada
31 |         System.out.print("\nIntroduzca la ruta de la versión
|         anterior: ");
32 |         String oldVersionPath = scanner.nextLine().trim(); // línea borrada
```

```
34 |         System.out.print("\nIntroduzca la ruta de la nueva versión:
|         ");
35 |         String newVersionPath = scanner.nextLine().trim(); // línea borrada
37 |         Set<String> oldFiles = collectJavaFiles(oldVersionPath); // línea borrada
38 |         Set<String> newFiles = collectJavaFiles(newVersionPath); // línea borrada
40 |         VersionComparator.ComparisonResult result =
|         VersionComparator.compare(oldFiles, newFiles);
42 |         printComparisonResults(result); // línea borrada
44 |         Checker checkerChain = createCheckerChain(); // línea borrada
45 |         CodeProcessor processor = new CodeProcessor(checkerChain); // línea borrada
47 |         List<FileStats> results = analyzeProject(projectPath,
|         processor);
48 |         if (!results.isEmpty()) { // línea borrada
49 |             printResults(results, projectPath); // línea borrada
50 |         } // línea borrada
52 |         System.out.println(""); // línea borrada
53 |         System.out.print("¿Desea analizar otra ruta? (y/n): "); // línea borrada
54 |         String respuesta = scanner.nextLine().trim().toLowerCase(); // línea borrada
55 |         if (!(respuesta.equals("y") || respuesta.equals("yes"))) { // línea borrada
56 |             System.out.println(""); // línea borrada
57 |             break; // línea borrada
58 |         } // línea borrada
59 |     } // línea borrada
61 |     scanner.close(); // línea borrada
62 | } // línea borrada
64 | private static Set<String> collectJavaFiles(String projectPath) { // línea borrada
65 |     Set<String> javaFiles = new HashSet<>(); // línea borrada
66 |     Path root = Paths.get(projectPath).normalize().toAbsolutePath(); // línea borrada
68 |     try { // línea borrada
```

```
69 | Files.walkFileTree(root, new SimpleFileVisitor<Path>() { // línea borrada
70 |     @Override // línea borrada
71 |     public FileVisitResult visitFile(Path file,
|     BasicFileAttributes attrs) {
72 |         if (file.toString().toLowerCase().endsWith(".java"))
|         {
73 |             String relativePath =
|             root.relativeTo(file).toString();
74 |             javaFiles.add(relativePath); // línea borrada
75 |         } // línea borrada
76 |         return FileVisitResult.CONTINUE; // línea borrada
77 |     } // línea borrada
78 | }); // línea borrada
79 | } catch (IOException e) { // línea borrada
80 |     System.err.println("Error: " + e.getMessage()); // línea borrada
81 | } // línea borrada
82 | return javaFiles; // línea borrada
83 | } // línea borrada
85 | private static void
| printComparisonResults(VersionComparator.ComparisonResult
| result) {
86 |     System.out.println("\n=== Comparación de Versiones ==="); // línea borrada
87 |     System.out.println("Archivos nuevos (" + result.addedFiles.size()
|     + "):");
88 |     result.addedFiles.forEach(p -> System.out.println(" [A] " + p)); // línea borrada
90 |     System.out.println("\nArchivos eliminados (" +
|     result.removedFiles.size() + "):");
91 |     result.removedFiles.forEach(p -> System.out.println(" [D] " +
|     p));
```

```
93 |     System.out.println("\nArchivos sin cambios (" +
|     result.unchangedFiles.size() + "):");
94 |     result.unchangedFiles.forEach(p -> System.out.println(" [=] " +
|     p));
95 | } // línea borrada
97 | private static Checker createCheckerChain() { // línea borrada
98 |     ParenthesesChecker parenthesesChecker = new ParenthesesChecker(); // línea borrada
99 |     LeftCurlyBraceChecker leftCurlyBraceChecker = new
|     LeftCurlyBraceChecker();
100 |     MultiInstanceChecker multiInstanceChecker = new
|     MultiInstanceChecker();
101 |     EndBreakChecker endBreakChecker = new EndBreakChecker(); // línea borrada
103 |     parenthesesChecker.setNext(leftCurlyBraceChecker); // línea borrada
104 |     leftCurlyBraceChecker.setNext(multiInstanceChecker); // línea borrada
105 |     multiInstanceChecker.setNext(endBreakChecker); // línea borrada
107 |     return parenthesesChecker; // línea borrada
108 | } // línea borrada
110 | private static List<FileStats> analyzeProject(String projectPath,
|     CodeProcessor processor) {
111 |     List<FileStats> results = new ArrayList<>(); // línea borrada
113 |     try { // línea borrada
114 |         Files.walkFileTree(Paths.get(projectPath), new
|         SimpleFileVisitor<Path>() {
115 |             @Override // línea borrada
116 |             public FileVisitResult visitFile(Path file,
|             BasicFileAttributes attrs) {
117 |                 if (file.toString().endsWith(".java")) { // línea borrada
118 |                     processFile(file, processor, results); // línea borrada
119 |                 } // línea borrada
```

```
120 |         return FileVisitResult.CONTINUE; // línea borrada
121 |     } // línea borrada
122 | }); // línea borrada
123 | } catch (IOException e) { // línea borrada
124 |     System.err.println("Error al analizar el proyecto: " +
|     e.getMessage());
125 | } // línea borrada
127 |     return results; // línea borrada
128 | } // línea borrada
130 | private static void processFile(Path file, CodeProcessor processor,
| List<FileStats> results) {
131 |     try { // línea borrada
132 |         FileStats stats = processor.processFile(file); // línea borrada
133 |         results.add(stats); // línea borrada
134 |     } catch (InvalidLineFormatException e) { // línea borrada
135 |         System.err.println("Error en " + file + ": " +
|         e.getMessage());
136 |     } catch (IOException e) { // línea borrada
137 |         System.err.println("No se pudo leer " + file); // línea borrada
138 |     } // línea borrada
139 | } // línea borrada
141 | private static void printResults(List<FileStats> results, String
| projectPath) {
142 |     int totalPhysicalLines = 0; // línea borrada
143 |     //int totalLogicalLines = 0; //It was discontinued due to
| changing requirements
144 |     int totalLines = 0; // línea borrada
146 |     String[] projectP = projectPath.split("\\\\"); // línea borrada
147 |     String projectName = projectP[projectP.length - 1]; // línea borrada
```



```
149 |     System.out.println(""); // línea borrada
150 |     System.out.println("Programa: " + projectName); // línea borrada
151 |     System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s | %-15s
|     |",
152 |         "Clase", "Metodos", "LOC f Clase", "Lineas Clase", "LOC f
|         Programa", "Lineas Programa");
153 |     System.out.println(""); // línea borrada
154 |     System.out.println("=".repeat(117)); // línea borrada
156 |     for (FileStats stats : results) { // línea borrada
158 |         totalPhysicalLines += stats.physicalLines(); // línea borrada
159 |         //totalLogicalLines += stats.logicalLines(); // línea borrada
160 |         totalLines += stats.lines(); // línea borrada
162 |         for(int i = 0; i < stats.classes().size(); i++) { // línea borrada
163 |             System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s
|             | %-15s |",
164 |                 stats.classes().get(i).getName(), // línea borrada
165 |                 stats.classes().get(i).getMethodCount(), // línea borrada
166 |                 stats.classes().get(i).getPhysicalLOC(), // línea borrada
167 |                 stats.classes().get(i).getLines(), // línea borrada
168 |                 "", // línea borrada
169 |                 ""); // línea borrada
170 |             System.out.println(""); // línea borrada
171 |         } // línea borrada
172 |     } // línea borrada
174 |     System.out.println("=".repeat(117)); // línea borrada
175 |     System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s | %-15s
|     |",
176 |         "", "", "", "", totalPhysicalLines, totalLines); // línea borrada
177 | } // línea borrada
```

178 | } // línea borrada

Archivo modificado: src/main/java/com/mantenimiento/azul/comparator/VersionComparator.java

```
1 | package com.mantenimiento.azul.comparator;
3 | import java.io.IOException; // línea añadida
4 | import java.nio.file.Files; // línea añadida
5 | import java.nio.file.Path; // línea añadida
6 | import java.nio.file.Paths; // línea añadida
7 | import java.util.Arrays; // línea añadida
8 | import java.util.HashSet;
9 | import java.util.Set;
11 | import com.mantenimiento.azul.model.ComparisonResult; // línea añadida
14 |     public static class ComparisonResult { // línea borrada
15 |         public Set<String> addedFiles = new HashSet<>(); // línea borrada
16 |         public Set<String> removedFiles = new HashSet<>(); // línea borrada
17 |         public Set<String> unchangedFiles = new HashSet<>(); // línea borrada
19 |     public static ComparisonResult compare(Set<String> version1,
|     Set<String> version2,
20 |         String oldBasePath, String
|         newBasePath) {
22 |     public static ComparisonResult compare(Set<String> version1,
|     Set<String> version2) {
25 |         Set<String> v1 = normalizePaths(version1);
26 |         Set<String> v2 = normalizePaths(version2);
28 |         result.removedFiles.addAll(v1); // línea borrada
29 |         result.removedFiles.removeAll(v2); // línea borrada
31 |         result.addedFiles.addAll(v2); // línea borrada
32 |         result.addedFiles.removeAll(v1); // línea borrada
34 |         result.unchangedFiles.addAll(v1); // línea borrada
```

```
35 |      result.unchangedFiles.retainAll(v2); // línea borrada
38 |      result.getRemovedFiles().addAll(v1); // línea añadida
39 |      result.getRemovedFiles().removeAll(v2); // línea añadida
41 |      result.getAddedFiles().addAll(v2); // línea añadida
42 |      result.getAddedFiles().removeAll(v1); // línea añadida
44 |      Set<String> possiblyUnchanged = new HashSet<>(v1); // línea añadida
45 |      possiblyUnchanged.retainAll(v2); // línea añadida
47 |      getModifedPaths(possiblyUnchanged, oldBasePath, newBasePath,
|      result);
52 |  private static void getModifedPaths(Set<String> possiblyUnchanged,
|  String oldBasePath, String newBasePath, ComparisonResult
|  result){
53 |      for (String relativePath : possiblyUnchanged) { // línea añadida
54 |          Path oldFile = Paths.get(oldBasePath, relativePath); // línea añadida
55 |          Path newFile = Paths.get(newBasePath, relativePath); // línea añadida
57 |          try { // línea añadida
58 |              byte[] oldBytes = Files.readAllBytes(oldFile); // línea añadida
59 |              byte[] newBytes = Files.readAllBytes(newFile); // línea añadida
61 |              if (Arrays.equals(oldBytes, newBytes)) { // línea añadida
62 |                  result.getUnchangedFiles().add(relativePath); // línea añadida
63 |              } else { // línea añadida
64 |                  result.getModifiedFiles().add(relativePath); // línea añadida
65 |              } // línea añadida
66 |          } catch (IOException e) { // línea añadida
67 |              System.err.println("Error comparando archivos: " +
|              relativePath + " - " + e.getMessage());
68 |          } // línea añadida
70 |      } // línea añadida
72 |  private static Set<String> normalizePaths(Set<String> paths) { // línea añadida
```

```
73 |     Set<String> normalized = new HashSet<>(); // línea añadida
74 |     for (String path : paths) { // línea añadida
75 |         normalized.add(path.replace("\\", "/")); // línea añadida
76 |     } // línea añadida
77 |     return normalized; // línea añadida
78 | } // línea añadida
79 | } // línea añadida
```